

MWW Project-Page Dashboard Operations Manual

June 2026

Contents

MWW Project-Page Dashboard	1
Stack	2
Repo layout	2
User roles	3
Features	3
Submitting a project	3
Drafts and approval	4
Publishing (admin only)	4
Editing published projects (admin only)	4
Deleting published projects (admin only)	4
Per-spoke + per-service + master archive pages	4
Project-page features (what gets generated)	5
YouTube video uploader (/videos)	5
Social-post helper (/projects/:id/social, admin-only)	6
Per-hub Google review QR codes (/review-qr)	6
Tech attribution	6
Team management (/users, admin-only)	6
Daily SQLite backup	6
PWA install	6
Configuration	7
First-time setup	7
Day-to-day operation	8
Deploy a code change	8
Publish flow (admin)	8
Edit a published project (admin)	8
Delete a published project (admin)	8
Add a tech (admin)	8
Connect YouTube channel (admin, one-time)	8
Set per-hub review URLs (admin, one-time)	8
Known scope limits	9

MWW Project-Page Dashboard

Mobile-first Node app that lets a tech submit a completed-job report (address, photos, price, customer review, before/after) and have it turned into a SOP-compliant static project page, with the matching spoke-page tile + archive pages + sitemap + footer column all updated and pushed to the mywindowwashing repo for cPanel to pull. Includes follow-on tooling for the same job: YouTube uploads, social-post helpers, and per-hub Google-review QR codes.

Deployed on a separate VPS at project.mywindowwashing.com — it never serves the public site itself, it only edits the static-site repo and runs `git push`. cPanel still needs a manual Deploy HEAD

Commit click for changes to go live.

Stack

- Node 20+ / Express, ESM modules
- SQLite (better-sqlite3) for the project log, users, and runtime settings
- Handlebars for both the dashboard UI and the project-page template
- sharp + heic-convert for phone-photo → 1200×800 .webp conversion (HEIC from iPhones is auto-detected and converted)
- cheerio for safe DOM edits to spoke + archive pages
- simple-git for commit + push, with retry-on-rejection
- Anthropic SDK (Claude) for narrative, FAQ, video metadata, and social-post text
- Google Places + Geocoding for address autocomplete + nearest-hub binding
- YouTube Data API v3 for video uploads
- node:crypto script for per-user password hashing
- Telegram Bot API for admin approval notifications
- PWA-installable on iOS + Android

Repo layout

```
src/  
  server.js Express bootstrap, session, view engine  
  routes/  
    auth.js /login, /logout, bootstrap admin  
    projects.js dashboard, new-project flow, preview, save, edit  
    publish.js render → spoke patch → archive → sitemap → commit + push  
    users.js team CRUD + photo upload (admin-only)  
    videos.js YouTube uploader page + OAuth connect/callback  
    social.js per-project social-post helper page  
    review-qr.js per-hub Google-review QR + setup  
  lib/  
    db.js sqlite schema, projects/users/settings CRUD  
    slug.js SERVICES, cities loader, hub binding, slug builder  
    photos.js sharp resize, HEIC detection, avatar 600×600  
    narrative.js Claude voice-tuned narrative generator  
    faq.js Claude FAQ generator (4 Q&A pairs)  
    render.js Handlebars view assembly + JSON-LD graph  
    spoke-update.js cheerio edits to <city>.html (forward + reverse)  
    archive.js per-spoke + per-service + master archive pages  
    sitemap.js sitemap.xml insert/refresh/remove  
    git.js simple-git wrapper with rebase-on-reject + retry  
    telegram.js admin approval ping  
    auth.js scrypt password hash/verify  
    youtube.js OAuth + resumable upload  
    video-meta.js Claude title/description/tags for YouTube  
    social-post.js Claude per-platform captions + hashtags  
    review-qr.js settings getters for hub review URLs  
    middleware/auth.js loadUser / requireAuth / requireAdmin  
  views/ express-handlebars dashboard UI  
templates/  
  project-page.hbs the canonical SOP project page, parameterised  
public/  
  manifest.json, icons PWA install  
scripts/
```

```
build-cities.js scan static site → cities.json (hub bindings)
sync-spoke-view-all.js refresh "View all N projects" footer links
backup-db.sh cron job: SQL dump → private backup repo
data/
  cities.json city → hub mapping (generated, committed)
  projects.db sqlite log (ignored from git, backed up nightly)
tmp/ working space for uploads + staged webps
```

User roles

The app supports two roles, stored on each user row:

Tech— submits projects, sees only their own drafts / pending / published on the dashboard. Cannot publish, edit, delete, manage team, or set up integrations. Can use the YouTube uploader, social-post helper, and review-QR page.

Admin— sees everything across the team. Approves & publishes tech submissions. Edits or deletes published projects (including delete from the live static site). Manages the team (add/remove users, set photos). Sets up integrations (YouTube channel connect, hub review URLs). **Cannot submit a project under their own name**— they're forced to pick a tech from a dropdown ("Submitting on behalf of") at the top of the new-project form.

First-run bootstrap: if the users table is empty when the server starts and DASHBOARD_PASSWORD is set, an admin user is created with that password. Otherwise nothing happens — you'll need to seed manually.

Features

Submitting a project

The new-project flow is two steps so the form stays narrow:

1. /new — pick a service: **Window Cleaning**, **Gutter Cleaning**, or **Power Washing**.
2. /new/details?service=... — service-specific form. The "Details" section changes per service (window types, gutter repair sub-grid, power-washing surface + material matrix). Common section across all three:
 - **1. Address** with Google Places autocomplete; the picked place's lat/lng is sent server-side
 - **2. Details** (per-service)
 - **3. Photos**— required before + after, up to 5 optional gallery extras. iPhone HEIC is auto-converted server-side. Photo inputs have no capture attribute so the OS picker shows Camera + Library + Files.
 - **4. Customer review** (optional) — name, Google/Yelp link, text
 - **5. What we did**— service-specific "Challenges encountered" checkbox grid + bullet-notes textarea + free-text **Other** extra (window-cleaning, gutter, and power all have an Other field at the bottom of their extras grid that opens a description input when ticked — the text flows to the AI narrative and appears as a Scope-of-Work tag on the published page)
 - **Submitting on behalf of**— admin-only card at the top, required; drops the picker for techs

The slug pattern is [service]-in-[city]-mm-dd-yy. Collisions prompt for a one-word descriptor that's appended.

The customer city is resolved separately from the hub: - City NAME shown on the page = the actual customer's city (Romeoville, Lemont, etc.) even when we don't have a spoke page for it - HUB binding = nearest spoke page (drives phone number on the page, Schema.org parentOrganization, and which spoke page gets a tile patched)

Drafts and approval

When the tech finishes the form and clicks Save, the row goes to status = 'draft'. A draft can be resumed from the dashboard's **Your drafts** card.

When the tech clicks **Save** on the preview page, the row moves to status = 'pending' and a Telegram message pings the admin with a "Review & publish" link. The Telegram ping is **skipped** when an admin is the one saving (no point notifying themselves).

Publishing (admin only)

POST /projects/:id/publish renders the project page template into the static-site repo and:

- Writes projects/<slug>.html + the before/after/gallery .webp files in projects/img/
- Patches the matching spoke page (<spoke>.html) — replaces the first .project-card with matching service that says "Coming soon", or prepends a new card and drops the oldest. Updates the "View all N projects in [City]" link in the footer.
- Patches the footer "Recent Projects" column site-wide
- Adds/refreshes the sitemap.xml entry
- Regenerates the affected archive pages:
 - **Master**/projects/index.html — every published project
 - **Per-service**/projects/<service>.html — all of one service
 - **Per-spoke**/projects/<spoke>.html — all from one spoke
- Commits as MWW Dashboard <noreply@...> and pushes
- Flips the row to status = 'published'

cPanel still has to be told to deploy.

Editing published projects (admin only)

Each published project tile shows **Edit** + **Social** + **Delete** buttons. The edit page (/projects/:id/edit):

- Lets admin change narrative, customer review, photos (replace before/after by uploading a new file), video URL, home type, price, street, bullet notes, and challenges
- Has  **Regenerate narrative** and  **Regenerate FAQ** buttons that re-call Claude with the current inputs
- Saves write the updated page back to the static-site repo and push. Spoke tile + archive pages are also re-rendered so the new title / thumbnail propagates.

Deleting published projects (admin only)

Hard-deletes everything tied to the project:

- The projects/<slug>.html file + all its .webp photos
- The spoke-page tile (.project-card removed; "View all" count recalculated)
- The sitemap entry
- Any archive pages it appeared on (regenerated)
- The DB row

A summary screen lists what was removed and reminds the admin to do the cPanel deploy.

Per-spoke + per-service + master archive pages

Whenever a project is published, deleted, or edited, three archive pages are kept in sync:

- /projects/index.html — every published project, chronological
- /projects/window-cleaning.html etc. — all of one service
- /projects/northbrook.html etc. — all from one spoke

Each is a self-contained HTML page with proper JSON-LD CollectionPage + BreadcrumbList + Item-List schema.

Project-page features (what gets generated)

For each project:

- Hero with title, date stamp, city, before/after slot, hub office
- Stats bar (windows cleaned, sq footage, etc., per service)
- **Trust signals bar**(Insured, Family Owned, 5 Reviews, Since 2003)
- Narrative — two paragraphs from Claude, technician byline with photo
- Before/After comparison
- Optional gallery (1–5 extras)
- Optional video embed (YouTube or Vimeo URL → iframe + JSON-LD VideoObject + thumbnail URL)
- Customer review block
- Scope of Work tag buttons → service pages
- Internal links: spoke page + city, neighboring projects
- **Related Projects**— 3 most recent from different cities/services, with after-photo thumbnails (placeholder fillers if fewer than 3 exist)
- **Nearby Cities**internal-link block
- Office Location map (GBP iframe URL preferred; Maps Embed API fallback)
- FAQ section (4 Q&A pairs from Claude, with FAQPage schema)
- Hub footer with phone + address + recent projects column
- Mobile sticky CTA bar (call + book)

JSON-LD graph emitted: WebPage + Article + Service + Review + ImageObject + VideoObject + FAQPage + Speakable + BreadcrumbList + parentOrganization → hub LocalBusiness.

YouTube video uploader (/videos)

Standalone page (any logged-in user). Tech / admin:

1. Picks an MP4/MOV file
2. Optionally picks a project to attach it to
3. Hits **Generate with AI**— Claude writes a search-friendly YouTube title (with city + service), description (grounded in the project's narrative + the project-page link + the right hub phone), and 10–15 tags
4. Reviews/edits the metadata, picks visibility (Public / Unlisted / Private), and clicks ► **Upload to YouTube**
5. Server streams the file to YouTube via the Data API v3 resumable upload protocol, then deletes the temp file
6. If linked to a project, the YouTube URL is saved to that project's video_url field. Published projects are re-rendered and pushed so the embed goes live immediately.

One-time admin setup at /youtube/connect — Google OAuth flow to authorize the company YouTube channel. The refresh token is stored in the settings table; from then on every upload by anyone on the team posts to that channel without any login.

Required env: YOUTUBE_CLIENT_ID, YOUTUBE_CLIENT_SECRET from a Google Cloud project with YouTube Data API v3 enabled and an OAuth Web client whose authorized redirect URI is <DASH-BOARD_BASE_URL>/youtube/callback.

nginx needs client_max_body_size 2G; and high proxy_*_timeout values for the upload to make it through.

Social-post helper (/projects/:id/social, admin-only)

Per-project page that:

- Generates Facebook / Instagram / LinkedIn captions tuned per platform (Facebook conversational + project link, Instagram punchy hook + line breaks + emoji, LinkedIn professional + no hashtags)
- Generates a hashtag set (used by FB + IG, skipped on LinkedIn)
- Shows a multi-select photo picker (before, after, gallery extras) — before + after pre-checked
- Per-platform card with:
 - **Copy caption + hashtags**— clipboard
 - **Download N photos**— opens each selected photo in a tab so you can long-press → Save to Photos on mobile
 - ↗ **Open Facebook / Instagram / LinkedIn**— composer in a new tab
- Character counter per platform with red warning above the limit
- 🔄 **Regenerate all captions** for a different draft

No platform API tokens, no App Review — manual posting stays the final step.

Per-hub Google review QR codes (/review-qr)

After finishing a job, the tech opens this page, picks the closest hub from a dropdown, and shows the customer a giant QR code on the phone screen. The customer scans → lands on the right Business Profile's review page in one tap. The dropdown pre-selects the hub from the tech's most recent project, so most flows are zero-click.

Admin sets up each hub's Google review short link once at /review-qr/setup. URLs are stored in the settings table — no deploy needed to edit. The setup page links out to `business.google.com` → Read reviews → Get more reviews and tells the admin to paste whatever Google shows (typically `https://g.page/r/<code>/review`).

Tech attribution

Every project page shows the submitting tech's photo + name as a byline on the narrative section. The dashboard lists also show "Submitted by {name}" so the office can see at a glance who filed what. Admin submissions always credit a tech (the one picked at the top of the form).

Team management (/users, admin-only)

CRUD users (techs + other admins). Photos are uploaded as 600×600 .webp, pushed to the static-site repo at `images/team/<username>.webp`, and used as the byline avatar on every project that tech submits.

Daily SQLite backup

`scripts/backup-db.sh` is a cron job that dumps the SQLite database as plain SQL and pushes it to a private GitHub repo using a dedicated deploy key (separate SSH config alias since one deploy key can't be on two repos). Push retries with exponential backoff on network flakes.

PWA install

`public/manifest.json` makes the dashboard installable as a home-screen app on iOS + Android. Brand-logo icon with "PROJECTS" overlay. iOS safe-area-inset is respected in the header padding so the navy bar sits below the notch.

Configuration

See .env.example for the canonical list. Highlights:

```
# Auth + sessions
DASHBOARD_PASSWORD=<bootstrap admin password on first run>
SESSION_SECRET=<random long string>

# HTTP
PORT=3000

# Anthropic – narrative, FAQ, video meta, social posts
ANTHROPIC_API_KEY=
ANTHROPIC_MODEL=claude-sonnet-4-6

# Google Maps – address autocomplete on the form
GOOGLE_MAPS_API_KEY=

# Google Geocoding – used only by scripts/build-cities.js
GOOGLE_GEOCODING_API_KEY=

# Static-site repo paths
SITE_REPO_PATH=/srv/mywindowwashing
SITE_REPO_BRANCH=master
SITE_REPO_PUSH=true

# Git identity
GIT_AUTHOR_NAME=MWW Dashboard
GIT_AUTHOR_EMAIL=dashboard@mywindowwashing.com

# Telegram approval ping
TELEGRAM_BOT_TOKEN=
TELEGRAM_CHAT_ID=
DASHBOARD_BASE_URL=https://project.mywindowwashing.com

# YouTube upload
YOUTUBE_CLIENT_ID=
YOUTUBE_CLIENT_SECRET=
YOUTUBE_REDIRECT_URI=
YOUTUBE_DEFAULT_PRIVACY=public
```

First-time setup

```
git clone <this repo>
cd mww-new-projects-app
cp .env.example .env
# Edit .env – see above.
npm install

# Clone the mywindowwashing static-site repo to SITE_REPO_PATH.
git clone git@github.com:d-g-websites/mywindowwashing.git /srv/mywindowwashing

# Build the city → hub lookup by scanning each spoke page's hub link.
# Rerun whenever spoke pages are added or hub assignments change.
npm run build-cities
```

Launch.

npm start

The app listens on PORT. Put it behind a TLS reverse proxy (nginx with `client_max_body_size 2G` for video uploads).

Bootstrap admin: on first start with an empty users table, an admin user is created with DASHBOARD_PASSWORD. Add techs via /users once you log in.

Day-to-day operation

Deploy a code change

```
cd /var/www/mww-dashboard-app
git pull
sudo systemctl restart mww-dashboard
(npm install only if package.json changed.)
```

Publish flow (admin)

1. Telegram ping → click the “Review & publish” link
2. Preview the page in the iframe
3. **Publish**— writes to the static-site repo and pushes
4. cPanel: Git Version Control → Update from Remote → Deploy HEAD Commit

Edit a published project (admin)

Dashboard → tile → **Edit** → make changes → **Save changes & update live site**. Same cPanel deploy click required.

Delete a published project (admin)

Dashboard → tile → **Delete** → confirm. Removes everything (HTML, photos, spoke tile, sitemap entry, affected archives, DB row). cPanel deploy still required to remove from the live site.

Add a tech (admin)

/users → **Add user** → username (lowercase), display name, role, password, optional photo. Photo is pushed to the static-site repo at `images/team/<username>.webp`.

Connect YouTube channel (admin, one-time)

/videos → ► **Connect YouTube channel** → sign in to the Google account that owns the brand account → on the “Choose a brand account” screen pick **My Window Washing** (not your personal channel) → **Allow**. The refresh token is stored; all team uploads route to that channel from then on.

Set per-hub review URLs (admin, one-time)

/review-qr/setup → paste the `https://g.page/r/<code>/review` URL for each hub from business.google.com → Read reviews → Get more reviews → **Save all**.

Known scope limits

- **Old published pages** carry whatever template they were rendered with. Template changes only affect newly generated pages. To update an existing page, open it in the dashboard and click Save (no changes needed — the save re-renders).
- **cPanel still needs a manual deploy click** for any push to take effect on the live site.
- **Google Business Profile posting** is no longer possible via API for small businesses (Google deprecated the endpoint in 2024). The review-QR page is the closest substitute.